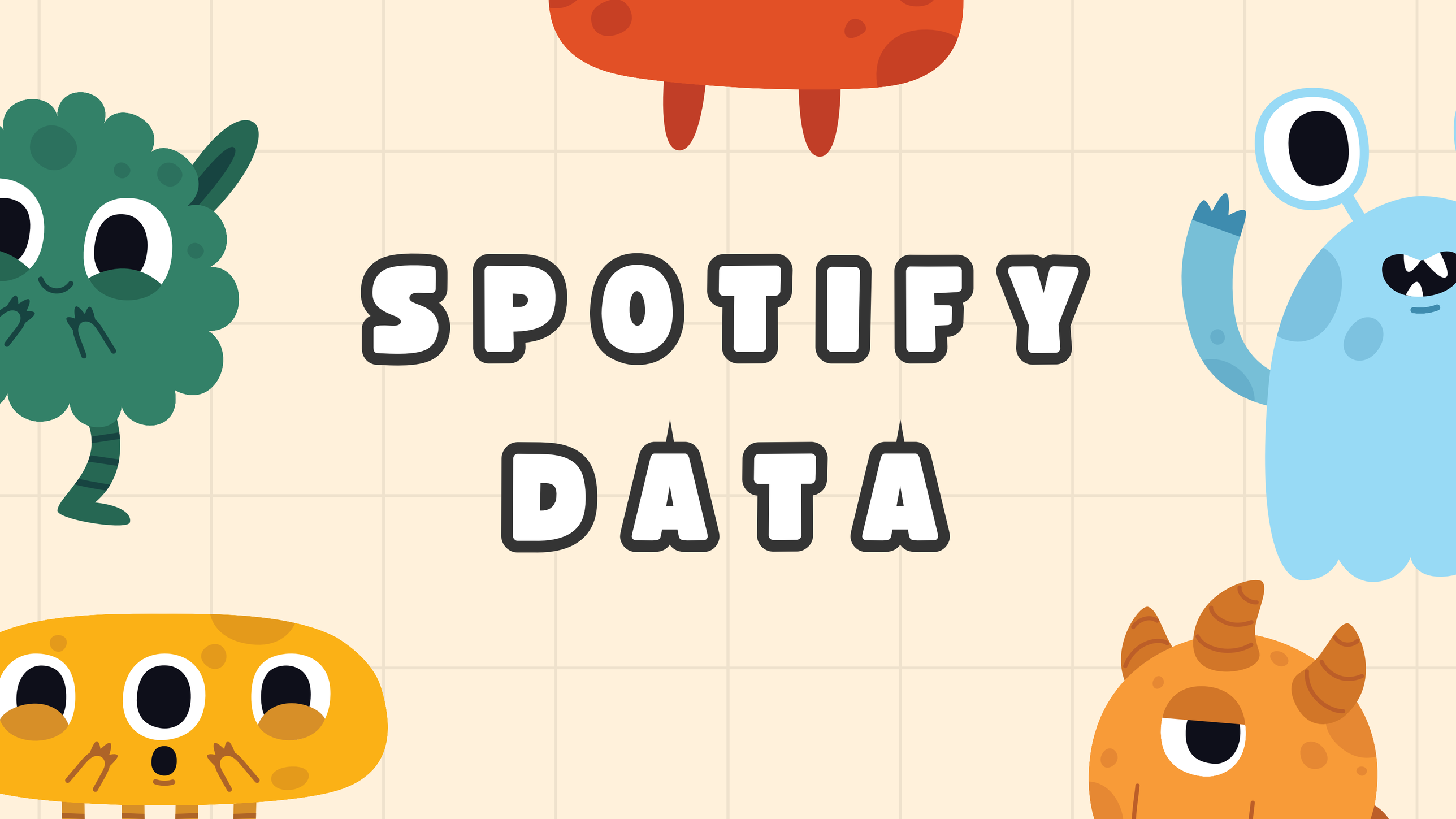
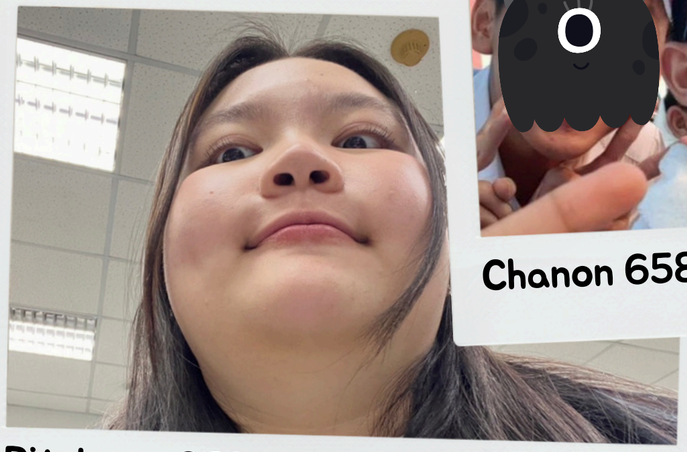
SPOTIFY DATA



GROUP MEMBERS

Natnicha 6580812



Pitchapa 6580065

Chanon 6581103

Supakorn 6581117

reducer.py mapper.py

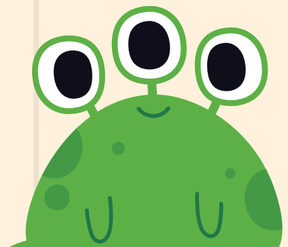
D: > oscar > reducer.py > ...

```python
#!/usr/bin/env python3
import sys
import re
from collections import defaultdict

counts = defaultdict(int)

#with open("/content/drive/MyDrive/Big Data Project(Spotify)/demofile.txt", mode ='r')as file:

for line in sys.stdin:

  word = line.strip()

  cleanWord = word.replace("\\", "")

  try:
      mylist = eval(cleanWord)
  except Exception:
      mylist = [cleanWord]

  #Check some unstructure format
  final_artists = []
  for entry in mylist:
    pieces = re.split(r'[\/,-]', entry.strip())
    for p in pieces:
        cleaned = p.strip()
        if cleaned:
            final_artists.append(cleaned)

  #Separate all the artist
  for artist in final_artists:
    counts[artist] += 1

for artist, count in counts.items():
  print(f"{artist:60}{counts[artist]}")
```

reducer.py mapper.py

D: > oscar > mapper.py > ...

```python
#!/usr/bin/env python3
import csv
import sys

reader = csv.reader(sys.stdin)
for index,row in enumerate(reader):
  if(index != 0):
    print(f"{row[2]}\n") #since in our .csv file the artists column is index 2
```



WHAT DATA WE DECIDED TO USE IN REGRESSION, CLASSIFICATION, AND CLUSTER ?

REGRESSION

LinearRegression

Regression

```
[ ]  from pyspark.sql import DataFrameNaFunctions
     from pyspark.ml.feature import VectorAssembler
```

```
     df = df.na.drop() #Dropping all rows with missing values
```

```
[ ]  df = df.withColumnRenamed("tempo", "label")
```

```
[ ]  # Select only numeric features
     from pyspark.sql.types import StringType

     featureColumns = [field.name for field in df.schema.fields
                       if not isinstance(field.dataType, StringType) and field.name != 'label']
```

REGRESSION

```
]  # Assemble features into vector
   from pyspark.ml.feature import VectorAssembler

   assembler = VectorAssembler(inputCols=featureColumns, outputCol="features")
   assembled = assembler.transform(df)
```

LinearRegression

```
[ ]  select_assembled = assembled.select("label", "features")

[ ]  (trainData, testData) = select_assembled.randomSplit([0.8,0.2], seed = 13234 )
```

```
[ ]  # Train Linear Regression
     lr = LinearRegression()

[ ]  model1 = lr.fit(trainData)
```

TRAIN-TEST SPLIT AND MODEL TRAINING

REGRESSION

```
# Evaluate
trainingSummary = model1.summary
trainingSummary.residuals.show()
print("RMSE: %f" % trainingSummary.rootMeanSquaredErr
print("r2: %f" % trainingSummary.r2)
```

```
numIterations: 0
objectiveHistory: [0.0]
+-------------------+
|          residuals|
+-------------------+
| -114.6509485914364|
|-112.23053736586584|
|-111.26141079563513|
|-110.77995100634153|
|-111.50217176440172|
|-120.37625477318004|
|-120.58642342968274|
|-119.38527128672278|
|-103.66155957345745|
|-107.75307375896281|
|-103.81281967300887|
|-103.41653856480674|
| -117.8987244289523|
|-117.10647205070843|
| -125.4804945786241|
|-105.41102460255244|
| -119.9349681200219|
|-123.51452478233522|
|-117.75773452334204|
|-120.37314646473926|
+-------------------+
only showing top 20 rows

RMSE: 29.169675
r2: 0.100024
```

Tempo

```
# Evaluate
trainingSummary = model1.summary
print("numIterations: %d" % trainingSummary.totalIterations)
print("objectiveHistory: %s" % str(trainingSummary.objectiveHi
trainingSummary.residuals.show()
print("RMSE: %f" % trainingSummary.rootMeanSquaredError)
print("r2: %f" % trainingSummary.r2)
```

```
numIterations: 0
objectiveHistory: [0.0]
+--------------------+
|           residuals|
+--------------------+
|-0.38018878528707756|
| -0.2649052313093643|
|-0.29862062594004213|
| -0.2799422286723072|
| -0.2904053836892313|
| -0.5005288537132908|
|  -0.504259924052394|
| -0.4812279302228599|
|-0.18834512776269818|
|  -0.269172032396064|
|-0.20467912361439655|
|-0.20364702486314368|
| -0.2263523317025007|
|-0.42506746254903316|
| -0.2830733329847579|
|-0.19692615665691493|
| -0.4585419003787039|
| -0.5383786007484685|
|  -0.449410800163186|
| -0.4857670216194563|
+--------------------+
only showing top 20 rows

RMSE: 0.124415
r2: 0.496841
```

Danceability

REGRESSION

```
[ ]  predictions = model1.transform(testData)
```

LinearRegression

PREDICTION

```
predictions.show()
```

Danceability

```
+------+--------------------+------------------+
| label|            features|        prediction|
+------+--------------------+------------------+
|   0.0|(14,[0,1,6,12],[6...| 0.2582056670015993|
|   0.0|[55693.0,1964.0,0...| 0.2853874758226742|
|   0.0|[60280.0,1966.0,0...| 0.3167233014686275|
|   0.0|[75000.0,2018.0,0...| 0.5078729726856495|
|   0.0|[93452.0,2016.0,0...|  0.453388076216942|
|   0.0|[104516.0,2017.0,...| 0.4124568507380566|
|   0.0|[158984.0,2017.0,...| 0.1515473686414055|
|   0.0|[159600.0,1937.0,...| 0.3429407257289583|
|   0.0|[170614.0,2017.0,...| 0.5236080117855371|
|   0.0|[180656.0,2017.0,...| 0.4626693441758474|
|   0.0|[192130.0,1948.0,...| 0.4055102042342549|
|0.0583|[600200.0,1962.0,...| 0.3132146060079113|
|0.0587|[272533.0,1954.0,...|0.26136160055245994|
|0.0596|[484013.0,2009.0,...|  0.364717741427552|
|0.0603|[304655.0,2009.0,...| 0.3602958294826839|
|0.0604|[219693.0,2005.0,...| 0.3874257596702493|
|0.0604|[232871.0,2013.0,...|0.40481732412899807|
|0.0608|[239547.0,1948.0,...| 0.2667799708221201|
|0.0613|[471368.0,1935.0,...|0.27409921842614327|
|0.0614|[416387.0,2000.0,...|0.35601767859171174|
+------+--------------------+------------------+
only showing top 20 rows
```

CLASSIFICATION

Logistic Regression

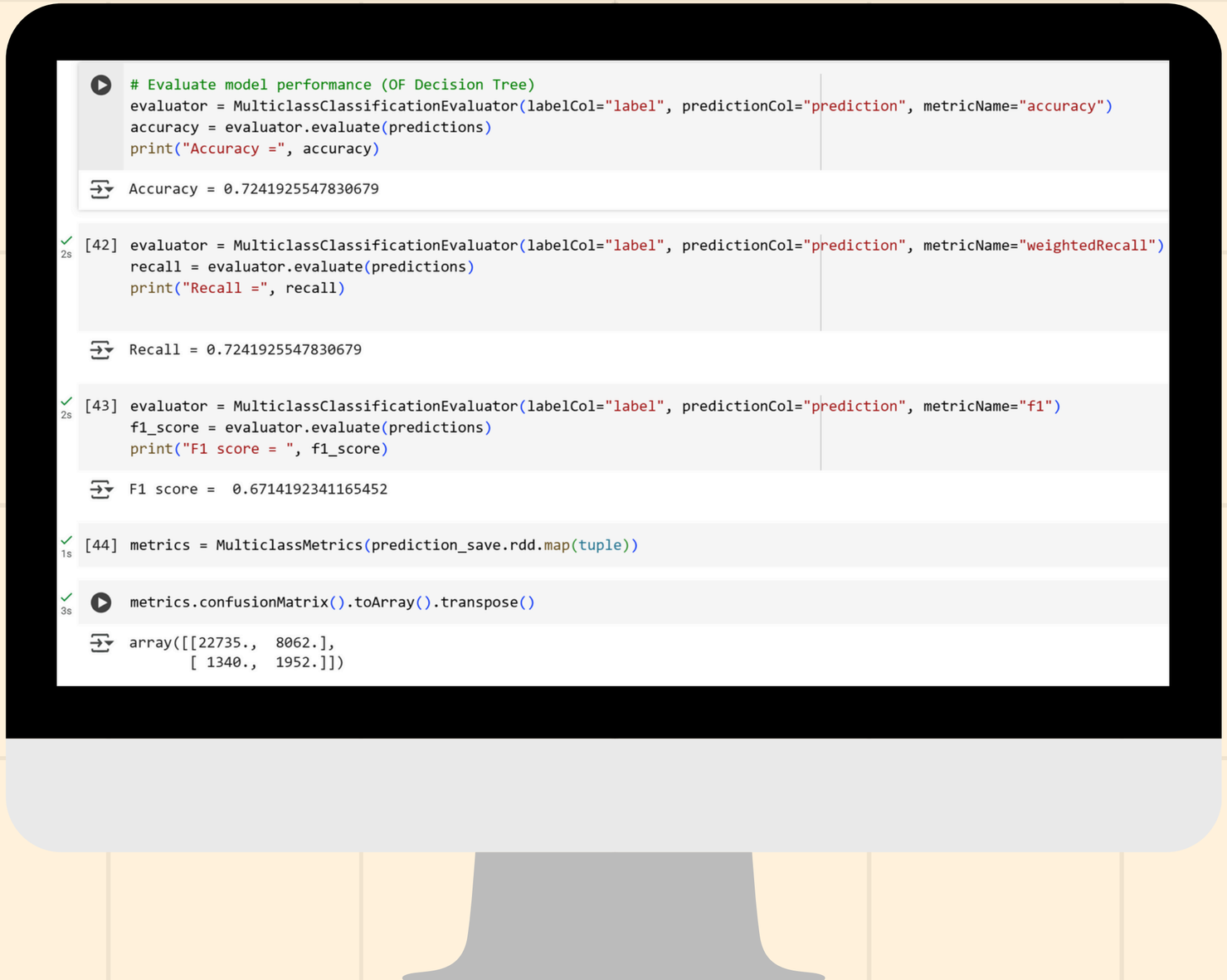
```
from pyspark.ml.classification import DecisionTreeClassifier,LogisticRegression
#Using Logistic Regression
lr = LogisticRegression(featuresCol="features", labelCol="label")
model = lr.fit(trainingData)

predictions = model.transform(testData)
```

```
#FROM Logistic Regression
predictions.select("features","rawprediction","probability","prediction", "label").show(15)
```

```
+--------------------+----------------+--------------------+----------+-----+
|            features|   rawprediction|         probability|prediction|label|
+--------------------+----------------+--------------------+----------+-----+
|[1989.0,0.117,0.4...|  [1092.0,527.0]|[0.67449042618900...|       0.0|  0.0|
|[1955.0,0.556,0.8...|   [101.0,325.0]|[0.23708920187793...|       1.0|  0.0|
|[2003.0,0.0948,0....|   [597.0,208.0]|[0.74161490683229...|       0.0|  0.0|
|[1979.0,0.055,0.3...|[11770.0,2583.0]|[0.82003762279662...|       0.0|  0.0|
|[2007.0,0.564,0.3...|   [240.0,103.0]|[0.69970845481049...|       0.0|  0.0|
|[1962.0,0.456,0.4...|[11770.0,2583.0]|[0.82003762279662...|       0.0|  0.0|
|[1973.0,0.847,0.4...|[20307.0,4838.0]|[0.80759594352754...|       0.0|  0.0|
|[1954.0,0.98,0.46...| [4486.0,2315.0]|[0.65960888104690...|       0.0|  1.0|
|[1963.0,0.426,0.6...|  [3983.0,844.0]|[0.82515019680961...|       0.0|  0.0|
|[1931.0,0.989,0.3...| [7868.0,4032.0]|[0.66117647058823...|       0.0|  0.0|
|[1995.0,0.124,0.6...|     [53.0,38.0]|[0.58241758241758...|       0.0|  0.0|
|[1954.0,0.869,0.3...|[20307.0,4838.0]|[0.80759594352754...|       0.0|  0.0|
|[2012.0,0.548,0.5...| [6712.0,2207.0]|[0.75255073438726...|       0.0|  1.0|
|[2001.0,0.042,0.6...| [6712.0,2207.0]|[0.75255073438726...|       0.0|  0.0|
|[1929.0,0.313,0.7...|[20307.0,4838.0]|[0.80759594352754...|       0.0|  0.0|
+--------------------+----------------+--------------------+----------+-----+
only showing top 15 rows
```

CLASSIFICATION

Logistic Regression



```
# Evaluate model performance (OF Decision Tree)
evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="accuracy")
accuracy = evaluator.evaluate(predictions)
print("Accuracy =", accuracy)
```

```
Accuracy = 0.7241925547830679
```

```
[42] evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="weightedRecall")
     recall = evaluator.evaluate(predictions)
     print("Recall =", recall)
```

```
Recall = 0.7241925547830679
```

```
[43] evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="f1")
     f1_score = evaluator.evaluate(predictions)
     print("F1 score = ", f1_score)
```

```
F1 score =  0.6714192341165452
```

```
[44] metrics = MulticlassMetrics(prediction_save.rdd.map(tuple))
```

```
metrics.confusionMatrix().toArray().transpose()
```

```
array([[22735.,  8062.],
       [ 1340.,  1952.]])
```

CLASSIFICATION

DecisionTreeClassifier

```
from pyspark.ml import Pipeline
dt = DecisionTreeClassifier(labelCol="label", featuresCol="features", maxDepth=10,minInstancesPerNode=20, impurity="entropy")
pipeline = Pipeline(stages=[dt])
model = pipeline.fit(trainingData)

predictions = model.transform(testData)
```

```
prediction_save=predictions.select("features","rawprediction","probability","prediction", "label").show()
```

```
+--------------------+---------------+--------------------+----------+-----+
|            features|   rawprediction|         probability|prediction|label|
+--------------------+---------------+--------------------+----------+-----+
|[1989.0,0.117,0.4...|  [1092.0,527.0]|[0.67449042618900...|       0.0|  0.0|
|[1955.0,0.556,0.8...|   [101.0,325.0]|[0.23708920187793...|       1.0|  0.0|
|[2003.0,0.0948,0....|   [597.0,208.0]|[0.74161490683229...|       0.0|  0.0|
|[1979.0,0.055,0.3...|[11770.0,2583.0]|[0.82003762279662...|       0.0|  0.0|
|[2007.0,0.564,0.3...|   [240.0,103.0]|[0.69970845481049...|       0.0|  0.0|
|[1962.0,0.456,0.4...|[11770.0,2583.0]|[0.82003762279662...|       0.0|  0.0|
|[1973.0,0.847,0.4...|[20307.0,4838.0]|[0.80759594352754...|       0.0|  0.0|
|[1954.0,0.98,0.46...| [4486.0,2315.0]|[0.65960888104690...|       0.0|  1.0|
|[1963.0,0.426,0.6...|  [3983.0,844.0]|[0.82515019680961...|       0.0|  0.0|
|[1931.0,0.989,0.3...| [7868.0,4032.0]|[0.66117647058823...|       0.0|  0.0|
|[1995.0,0.124,0.6...|     [53.0,38.0]|[0.58241758241758...|       0.0|  0.0|
|[1954.0,0.869,0.3...|[20307.0,4838.0]|[0.80759594352754...|       0.0|  0.0|
|[2012.0,0.548,0.5...| [6712.0,2207.0]|[0.75255073438726...|       0.0|  1.0|
|[2001.0,0.042,0.6...| [6712.0,2207.0]|[0.75255073438726...|       0.0|  0.0|
|[1929.0,0.313,0.7...|[20307.0,4838.0]|[0.80759594352754...|       0.0|  0.0|
|[1954.0,0.751,0.7...|     [49.0,10.0]|[0.83050847457627...|       0.0|  1.0|
|[1940.0,0.729,0.6...| [7868.0,4032.0]|[0.66117647058823...|       0.0|  1.0|
|[1958.0,0.912,0.6...| [7868.0,4032.0]|[0.66117647058823...|       0.0|  0.0|
|[1990.0,0.00563,0...|[11770.0,2583.0]|[0.82003762279662...|       0.0|  1.0|
|[2008.0,0.213,0.8...| [6712.0,2207.0]|[0.75255073438726...|       0.0|  1.0|
+--------------------+---------------+--------------------+----------+-----+
only showing top 20 rows
```

CLASSIFICATION

DecisionTreeClassifier

```
# Evaluate model performance (OF Decision Tree)
evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="accuracy")
accuracy = evaluator.evaluate(predictions)
print("Accuracy =", accuracy)
```

```
Accuracy = 0.7241925547830679
```

```
evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="weightedRec
recall = evaluator.evaluate(predictions)
print("Recall =", recall)
```

```
Recall = 0.7241925547830679
```

```
evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="f1")
f1_score = evaluator.evaluate(predictions)
print("F1 score = ", f1_score)
```

```
F1 score =  0.6714192341165452
```

```
metrics = MulticlassMetrics(prediction_save.rdd.map(tuple))
```

```
metrics.confusionMatrix().toArray().transpose()
```

```
array([[22735.,  8062.],
       [ 1340.,  1952.]])
```

SCORE RESULT COMPARATION

LogisticRegression

```
prediction_save=predictions.select("prediction", "label")
```

```
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
from pyspark.mllib.evaluation import MulticlassMetrics
```

```
evaluator = MulticlassClassificationEvaluator(
    labelCol="label", predictionCol="prediction", metricName="accuracy")
```

```
accuracy = evaluator.evaluate(predictions)
print("Accuracy = %g " % (accuracy))
```

```
Accuracy = 0.710258
```

```
# Evaluate model performance (OF LogisticRegression)
evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="predictio
recall = evaluator.evaluate(predictions)
print("Recall =", recall)
```

```
Recall = 0.7102584411393704
```

```
evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="predictio
f1_score = evaluator.evaluate(predictions)
print("F1 score = ", f1_score)
```

```
F1 score =  0.6020256546310786
```

DecisionTreeClassifier

```
prediction_save=predictions.select("prediction", "label")
```

```
evaluator = MulticlassClassificationEvaluator(
    labelCol="label", predictionCol="prediction", metricName="accuracy")
```

```
accuracy = evaluator.evaluate(predictions)
print("Accuracy = %g " % (accuracy))
```

```
Accuracy = 0.724193
```

```
# Evaluate model performance (OF Decision Tree)
evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricNa
accuracy = evaluator.evaluate(predictions)
print("Accuracy =", accuracy)
```

```
Accuracy = 0.7241925547830679
```

```
evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricNa
recall = evaluator.evaluate(predictions)
print("Recall =", recall)
```

```
Recall = 0.7241925547830679
```

```
evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricNa
f1_score = evaluator.evaluate(predictions)
print("F1 score = ", f1_score)
```

```
F1 score =  0.6714192341165452
```

OTHERS PERDICTION COMPARATION

Predict Mode (2 Classes)

```
[48] accuracy = evaluator.evaluate(predictions)
     print("Accuracy = %g " % (accuracy))

     Accuracy = 0.710141

[50] evaluator = MulticlassClassificationEvaluator(label
     recall = evaluator.evaluate(predictions)
     print("Recall =", recall)

     Recall = 0.7101411012350025

     evaluator = MulticlassClassificationEvaluator(label
     f1_score = evaluator.evaluate(predictions)
     print("F1 score = ", f1_score)

     F1 score =  0.6016204421827177
```

Predict Popularity (101 Classes)

```
[ ] accuracy = evaluator.evaluate(predictions)
    print("Accuracy = %g " % (accuracy))

    Accuracy = 0.168647

[41] # Evaluate model performance (OF LogisticRegression)
     evaluator = MulticlassClassificationEvaluator(labelCo
     recall = evaluator.evaluate(predictions)
     print("Recall =", recall)

     Recall = 0.1686467775528763

[42] evaluator = MulticlassClassificationEvaluator(labelCo
     f1_score = evaluator.evaluate(predictions)
     print("F1 score = ", f1_score)

     F1 score =  0.08755216479167591
```

CLUSTER

K-means with Raw features

K is the number of cluster that we want our model to capture

since our model need to predict keys which has 0-11 key. SO OUR K WILL BE 12

```
[34] number_of_clusters = 12
```
0s

```
[35] from pyspark.ml.clustering import KMeans
     kmeans = KMeans(k = number_of_clusters)
```
0s

```
[36] model = kmeans.fit(assembled)
```
36s

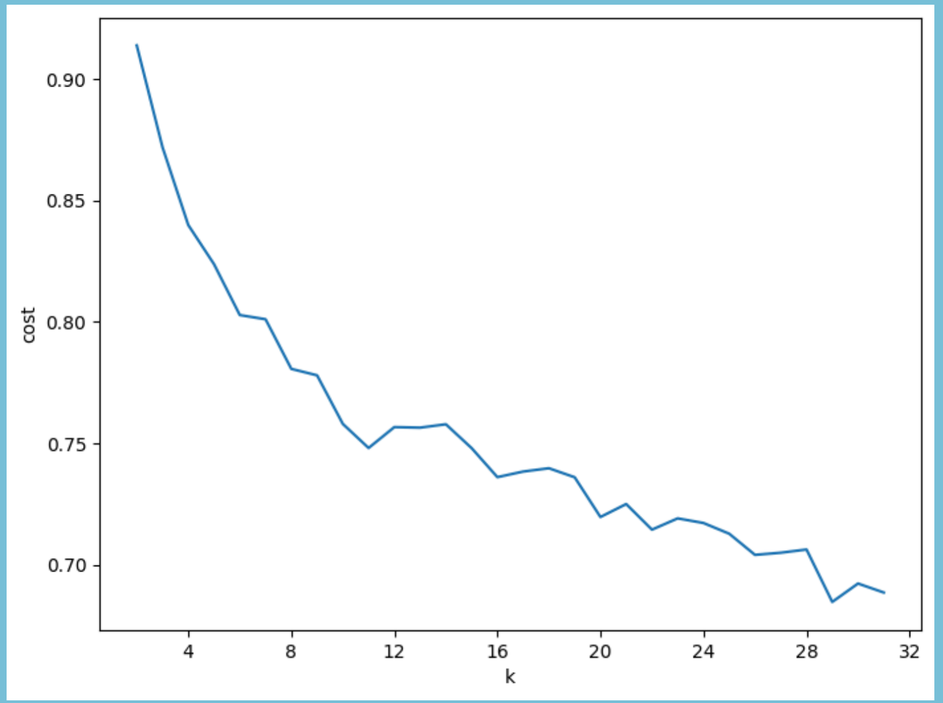
CLUSTER

K-means with Raw features

Data Evaluate the key

9m

```
import numpy as np
# Evaluate clustering by computing Silhouette score

cost = np.zeros(31)
for k in range(2,32):
    kmeans = KMeans()\
            .setK(k)\
            .setSeed(1)\

    model = kmeans.fit(elbowset)
    prediction=model.transform(elbowset)
    evaluator = ClusteringEvaluator()
    silhouette = evaluator.evaluate(prediction)
    cost[k-1] = silhouette
```



Elbow point around (k = 4)

CLUSTER

K-means with Raw features

```
[40] from pyspark.ml.evaluation import ClusteringEvaluator

     evaluator = ClusteringEvaluator()
     cost = evaluator.evaluate(predictions)
```

8s

```
print(cost)
```

0s

```
0.7553402387171909
```

This means each point is much closer to its own cluster than to the nearest other cluster.
In other words, High cohesion and High separation.

CLUSTER

K-means with standardized features

Scale the input data with 0 mean unit(1) variance

3s

```
from pyspark.ml.feature import StandardScaler

scaler = StandardScaler(inputCol="features", outputCol="features_std", withStd=True, withMean=True)
scalerModel = scaler.fit(assembled)
scaledData = scalerModel.transform(assembled)
```

0s

```
[51] kmean = KMeans(k = number_of_clusters,featuresCol="features_std")
```

32s

```
[52] model_std = kmean.fit(scaledData)
```

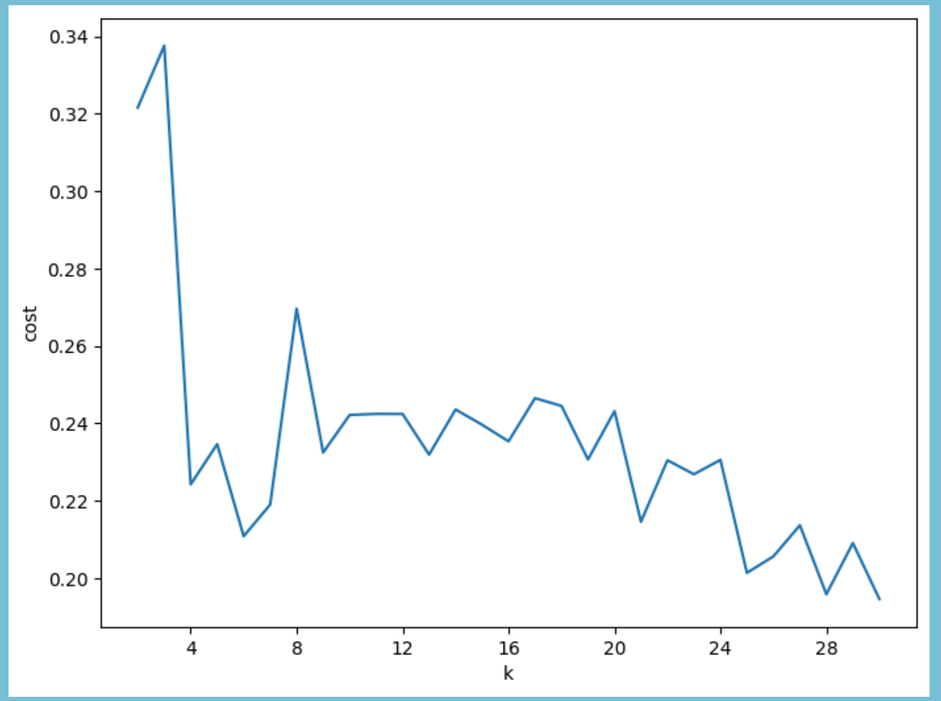
CLUSTER

K-means with standardized features

```
[56] elbowset = scaledData.select("features_std")
     elbowset.persist()
```

```
DataFrame[features_std: vector]
```

```
# Evaluate clustering by computing Silhouette score
cost = np.zeros(30)
for k in range(2,31):
    kmeans = KMeans(featuresCol="features_std")\
            .setK(k)\
            .setSeed(1)

    model = kmeans.fit(elbowset)
    prediction=model.transform(elbowset)
    evaluator = ClusteringEvaluator(featuresCol="features_std")
    silhouette = evaluator.evaluate(prediction)
    cost[k-1] = silhouette
```



Elbow point around (k = 4)

CLUSTER

K-means with standardized features

```
[55] evaluator = ClusteringEvaluator()
     cost = evaluator.evaluate(prediction_std)
     print(cost)
```

8s

```
-0.2678817570261831 6
```

This means points are closer to a neighboring cluster than to their own. Resulting in overlap between 2 cluster.

CLUSTER

Why standardized features are worse than raw features in this dataset

- Standardized features might drop some significant cluster-signal
- We do standardize only when the features are on wildly different units.

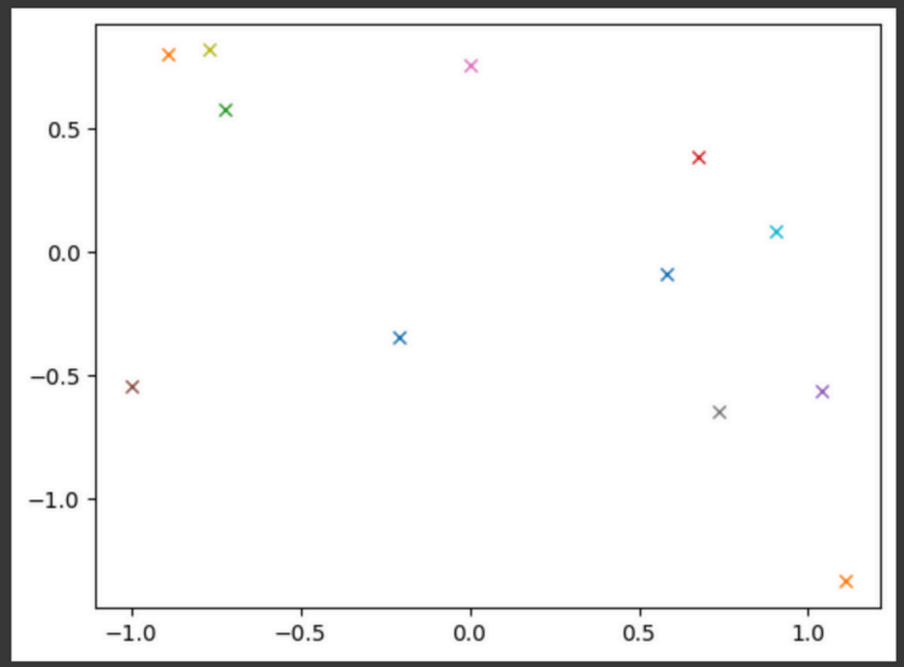
Most of the Spotify data is not very different in term of unit.

acousticness	danceability	energy	instrumentalness	liveness	loudness	speechiness	tempo	valence	mode	key	popularity	explicit	artists_index
0.995	0.708	0.195	0.563	0.151	-12.428	0.0506	118.469	0.779	1	10	0	0	1478.0
0.994	0.379	0.0135	0.901	0.0763	-28.454	0.0462	83.972	0.0767	1	8	0	0	166.0
0.604	0.749	0.22	0.0	0.119	-19.924	0.929	107.177	0.88	0	5	0	0	281.0
0.995	0.781	0.13	0.887	0.111	-14.734	0.0926	108.003	0.72	0	1	0	0	1.0
0.99	0.21	0.204	0.908	0.098	-16.829	0.0424	62.149	0.0693	1	11	1	0	65.0
0.995	0.424	0.12	0.911	0.0915	-19.242	0.0593	63.521	0.266	0	6	0	0	2348.0
0.956	0.444	0.197	0.435	0.0744	-17.226	0.04	80.495	0.305	1	11	0	0	443.0
0.988	0.555	0.421	0.836	0.105	-9.878	0.0474	123.31	0.857	1	1	0	0	1478.0
0.995	0.683	0.207	0.206	0.337	-9.801	0.127	119.833	0.493	0	9	0	0	10.0
0.846	0.674	0.205	0.0	0.17	-20.119	0.954	81.249	0.759	1	9	0	0	281.0
0.994	0.376	0.0719	0.883	0.196	-21.849	0.0352	141.39	0.0393	0	10	0	0	2262.0
0.989	0.17	0.0823	0.911	0.0962	-30.107	0.0317	85.989	0.346	0	10	1	0	65.0
0.99	0.359	0.0435	0.899	0.109	-20.858	0.0424	96.645	0.042	1	7	0	0	2646.0
0.992	0.311	0.0107	0.883	0.0954	-35.648	0.0556	78.98	0.216	1	5	0	0	166.0
0.977	0.335	0.105	0.84	0.231	-16.049	0.0716	80.204	0.406	0	5	0	0	6586.0
0.991	0.319	0.00593	6.35E-5	0.0691	-25.789	0.051	79.831	0.169	0	7	0	0	2758.0
0.996	0.319	0.155	0.917	0.126	-18.728	0.036	66.947	0.0488	1	4	0	0	135.0
0.994	0.787	0.156	0.659	0.11	-14.056	0.157	117.167	0.849	0	4	0	0	6228.0
0.993	0.569	0.15	0.0015	0.106	-15.238	0.0474	76.93	0.596	1	5	0	0	7017.0
0.992	0.763	0.132	0.0693	0.112	-13.002	0.0886	111.679	0.832	1	4	0	0	1.0

CLUSTER

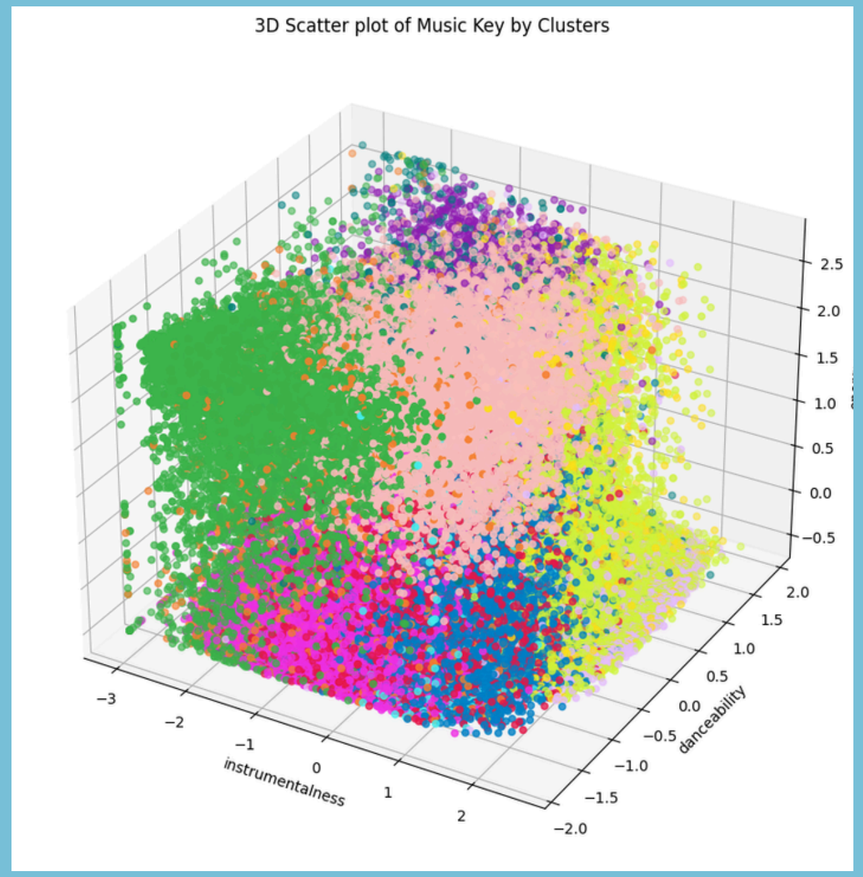
Visualize the cluster centers

```
for k in range(0,12):
    plt.plot(centers[k][0],centers[k][1],'x')
```



CLUSTER

Visualize by Plotting (Raw Features)



CLUSTER

Predict Key (12 Clusters)

- Raw features

```
[40] from pyspark.ml.evaluation import ClusteringEvaluator

     evaluator = ClusteringEvaluator()
     cost = evaluator.evaluate(predictions)

     print(cost)

     0.7553402387171909
```

- Standardized features

```
[55] evaluator = ClusteringEvaluator()
     cost = evaluator.evaluate(prediction_std)
     print(cost)

     -0.26788175702618316
```

Predict Mode (2 Clusters)

- Raw features

```
[70] from pyspark.ml.evaluation import ClusteringEvaluator

     evaluator = ClusteringEvaluator()
     cost = evaluator.evaluate(predictions)

[71] print(cost)

     0.913728119132859
```

- Standardized features

```
evaluator = ClusteringEvaluator()
cost = evaluator.evaluate(prediction_std)
print(cost)

-0.011155967304332923
```

CONCLUSION

Regression

Tempo Model Performance
- RMSE: 29.17
- R^2 Score: 0.10 (Only 10% of variation)

- Why so low?
- Tempo is genre/artist-dependent
- Numerical features may not capture stylistic elements

Danceability Model Performance
- RMSE: 0.12
- R^2 Score: 0.49 (Explains 49% of variation)

- Why better?
- Danceability is derived from other features
- Strong correlation with:
  - Energy
  - Valence
  - Rhythm-related metrics

Classification

Mode Model Performance
- The Decision Tree Classifier achieves higher overall performance compared to Logistic Regression on this dataset, making it the preferable model for this task

Popularity Model Performance
- All of the score in Decision Tree Classifier are lower than the mode model performance. This happens because each class of the popularity has a imbalance class.
- Reason: Class Imbalance
  - Popularity levels are not evenly distributed
  - Rare classes are underrepresented
  - Models may struggle to learn them, pulling down overall accuracy

Cluster

K-mean with Raw features in Key
- K-mean with Raw features' cost : 0.76
- K-mean with standardized features' cost : -0.2

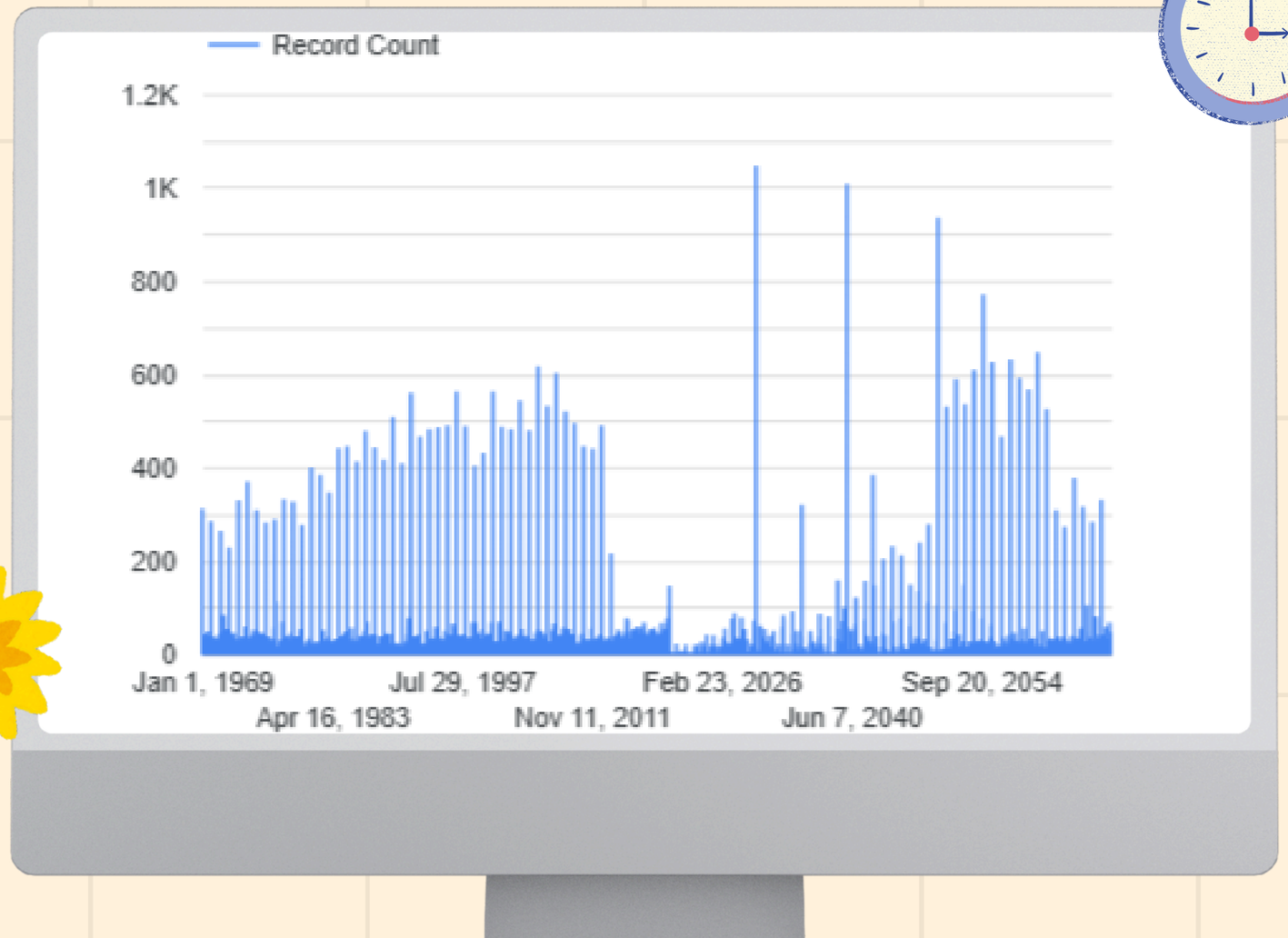
Since negative values aren't valid in standard K-Means cost, we conclude that clustering worked better without standardizing the features in this case.
Predict Key may be more informative, despite lower score, since it allows more complex segmentation

K-mean with Raw features in Mode
Even though the cost on the predict mode is higher than Key, this is because the number of clusters on the Mode side has only 2.
Silhouette score often favors very small k.

If you're optimizing purely for clustering quality: Predict Mode performs best numerically (Silhouette = 0.91), but Beware of over-simplification, it might be too coarse

VISUALIZATION

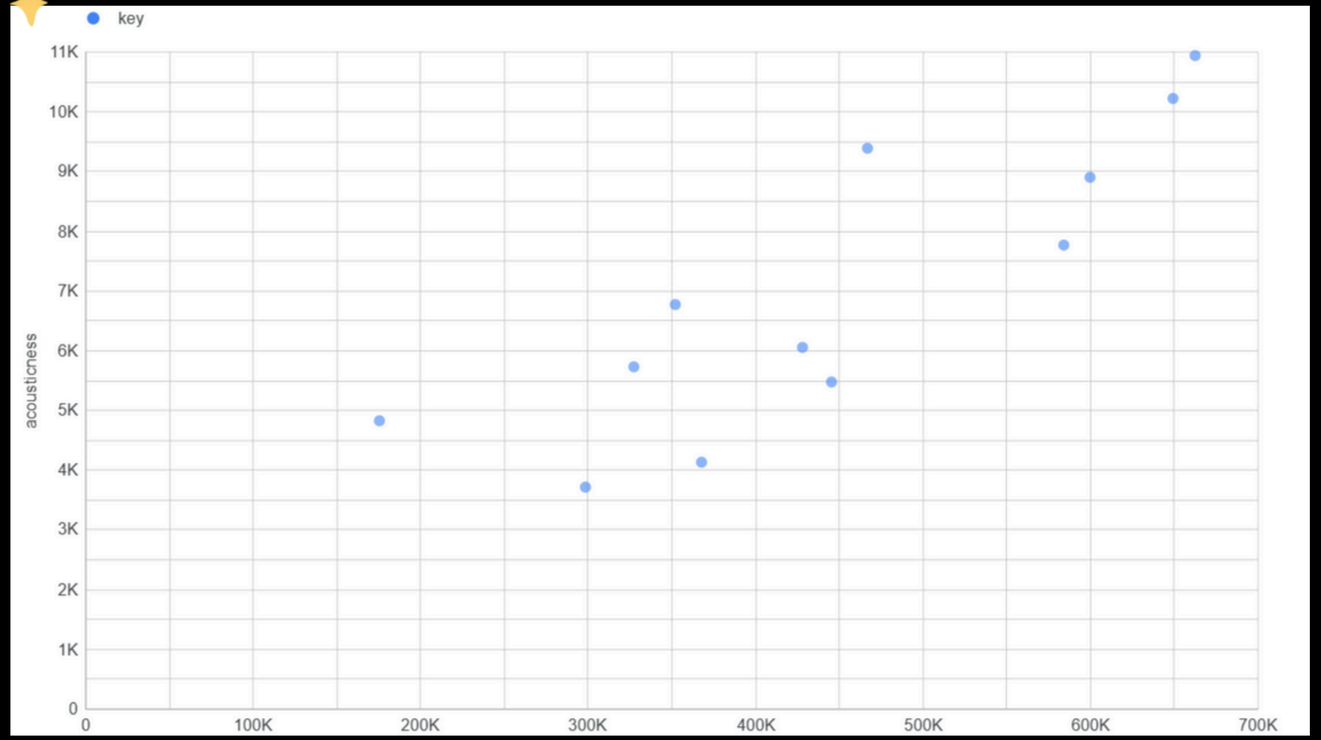
Record SeriesChart

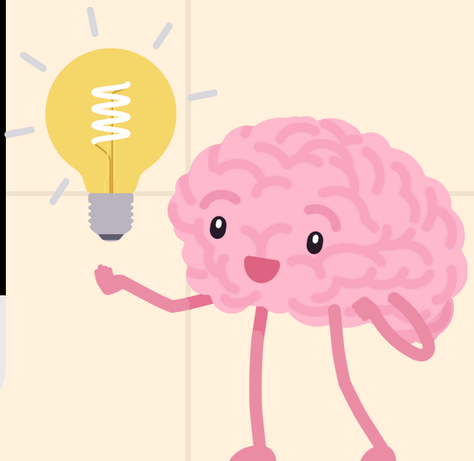


VISUALIZATION

Scatter Chart of Key

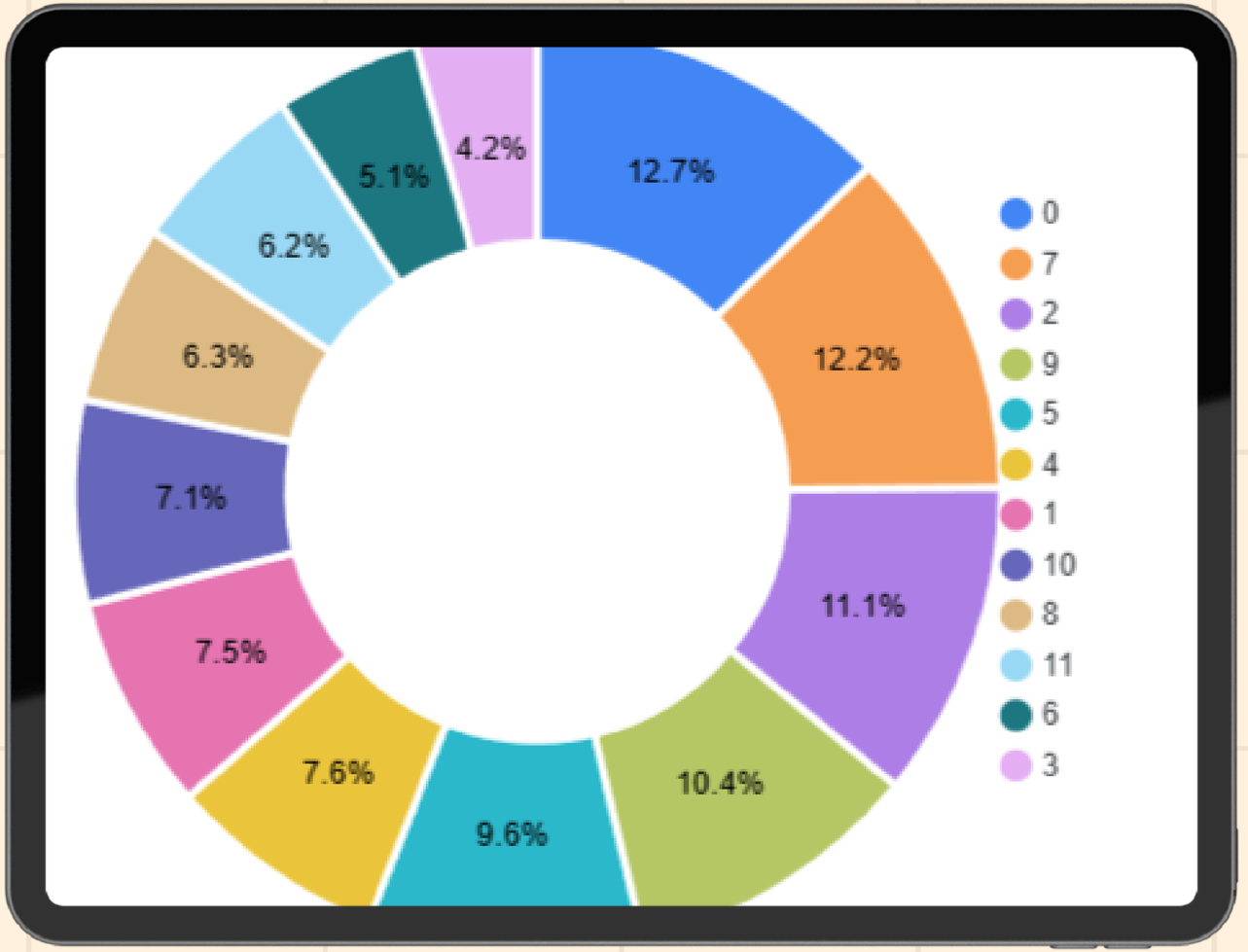
X: Popularity, Y: Acousticness

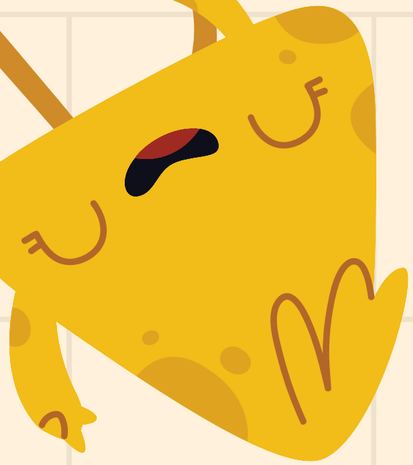


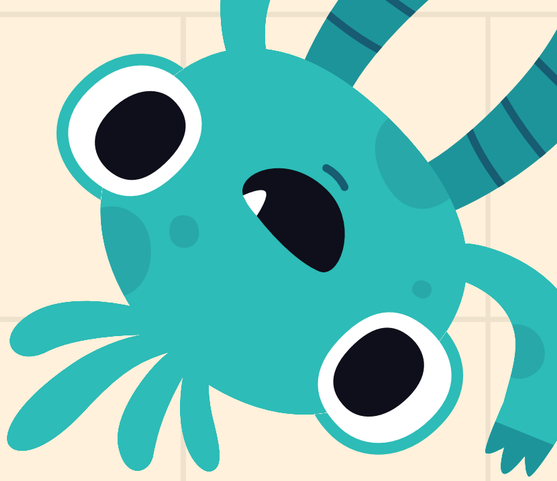


VISUALIZATION

Pie Chart of a key







THANK YOU